

CS222

Operating Systems

Lecture 12

Kernel Memory and

Dynamic Memory Allocation

(Section 100001)

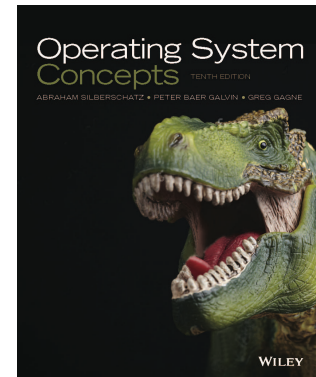
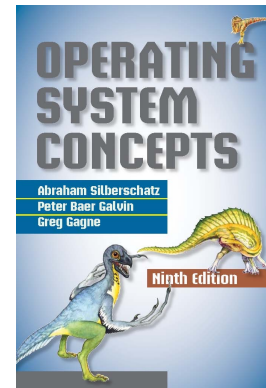
ผศ. ดร. กษิธิศ ชาญเขียว

ckasidit@tu.ac.th

Textbook

- Avi Silberschatz, Peter B. Galvin and Greg Gagne; Operating System Concepts, 9th Edition; John Wiley & Sons, Inc; 2012; ISBN 978-1118063330

- Chapter 10 (10th edition)
Virtual Memory



- Original Slides
- <https://www.os-book.com/OS9/slide-dir/index.html>
- <https://www.os-book.com/OS10/slide-dir/index.html>



การจัดการ **Memory** ของ **Linux Kernel**



Kernel Memory

- Load and run UEFI
- UEFI load Kernel
- UEFI uses paging using identity-mapped memory translation
- Kernel runs and inherit identity-mapped memory access
- Kernel switches MMU to paging using page table
- Linux Virtual Memory Layout



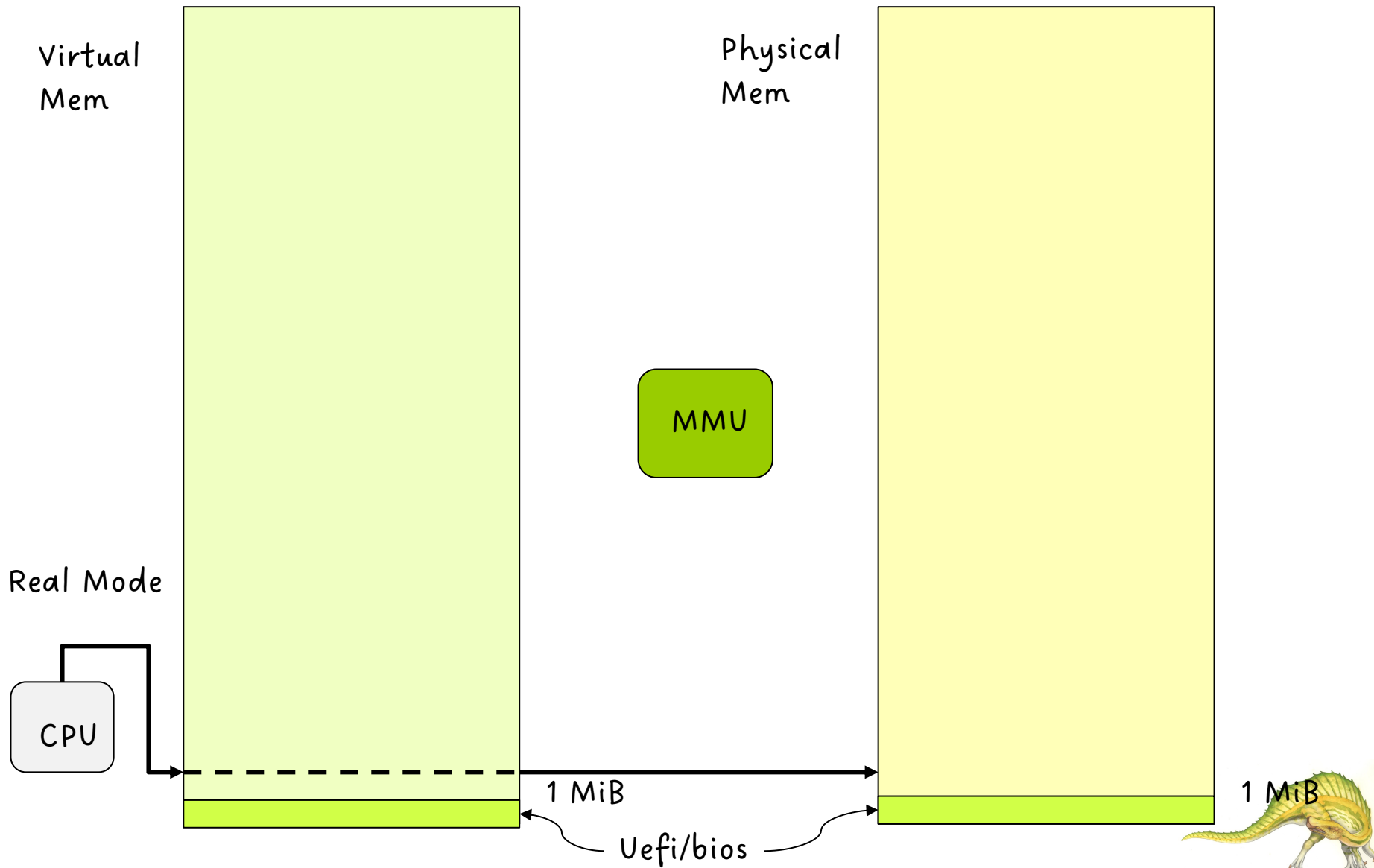
ขั้นตอน 1. โหลด รัน UEFI

- เมื่อคอมพิวเตอร์เริ่มต้นการประมวลผล เรียกว่า Setup Phase ซีพียูจะทำงานอยู่ใน **Real Mode** (https://en.wikipedia.org/wiki/Real_mode) ซึ่งเข้าถึงหน่วยความจำ Physical Memory โดยตรง
- ซีพียูจะโหลดโปรแกรม UEFI จาก Flash Storage Chip เข้าสู่หน่วยความจำและประมวลผล UEFI
- UEFI ตรวจสอบความถูกต้องของอุปกรณ์ในคอมพิวเตอร์ สร้าง Global Descriptor Table (GDT) และ Interrupt Descriptor Table (IDT) และซีพียูเป็น Long Mode (https://en.wikipedia.org/wiki/Long_mode) แบบ Identity-mapped paging
- อ้างอิง: <https://wiki.osdev.org/UEFI>





UEFI starts





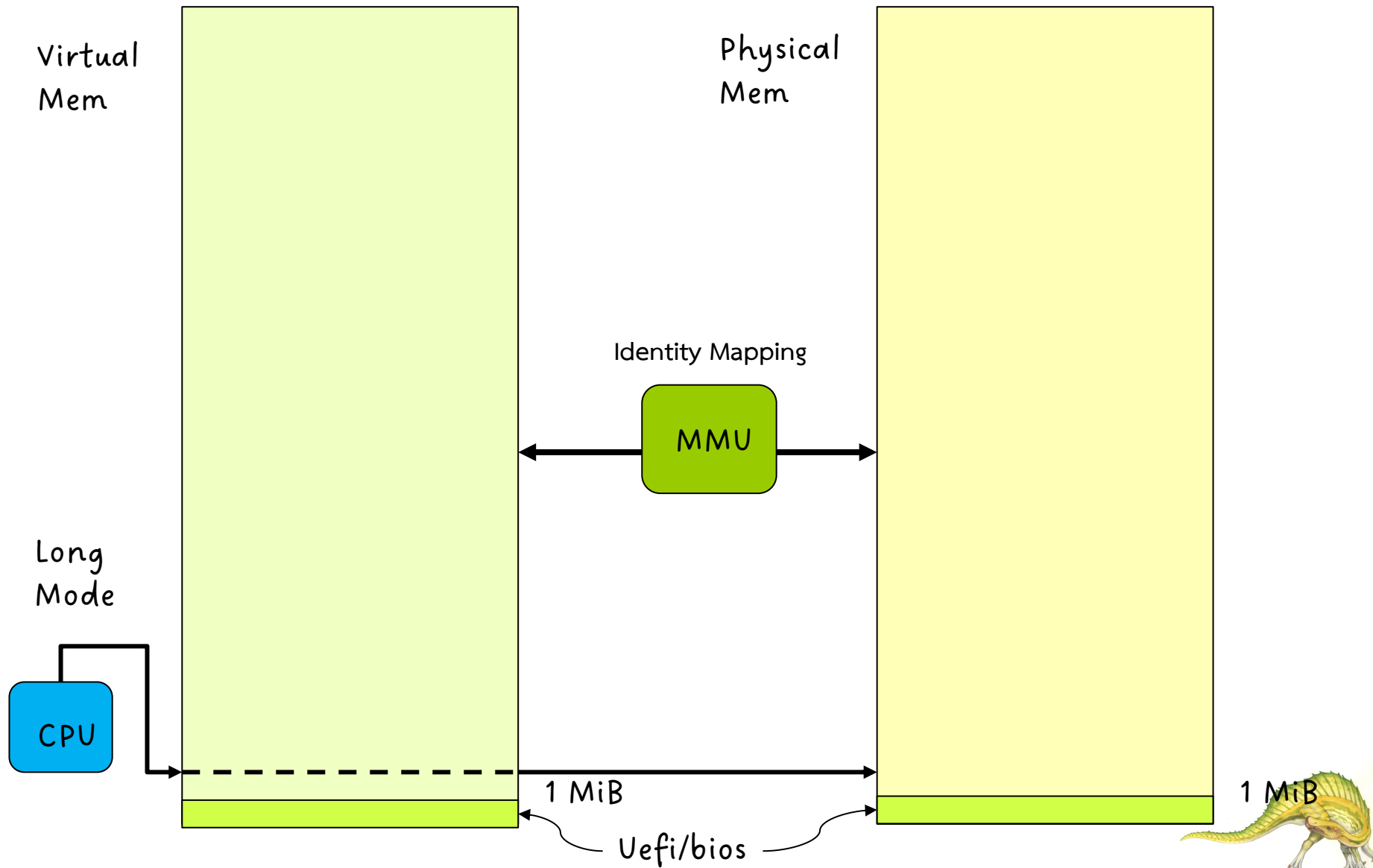
ขั้นตอน 2. UEFI โหลด Kernel

- UEFI กำหนดให้ CPU เข้าถึง Memory แบบ Identity-Mapped ซึ่งทำให้ MMU แปลงค่า Virtual Address (VA) ไปเป็น Physical Address (PA) แบบ $VA == PA$
- UEFI โหลด Driver ของอุปกรณ์เข้าสู่ Memory แล้วแสดงค่าสถานะของอุปกรณ์ออกสู่หน้าจอ (และรันโปรแกรม Interface เพื่อรับการกำหนดค่า Configuration ของฮาร์ดแวร์และ boot order จากผู้ใช้ ถ้าผู้ใช้ต้องการ)
- ในกรณีที่ผู้ใช้ติดตั้งโปรแกรม Grub (เช่น เพื่อเลือก OS kernel) UEFI จะโหลด Grub แล้ว Grub โหลด OS Kernel เข้าสู่ Memory
- ในกรณีที่ผู้ใช้ใช้ EFI boot stub โปรแกรม UEFI สามารถโหลด OS Kernel เข้าสู่ Memory ได้





UEFI uses identity-mapped





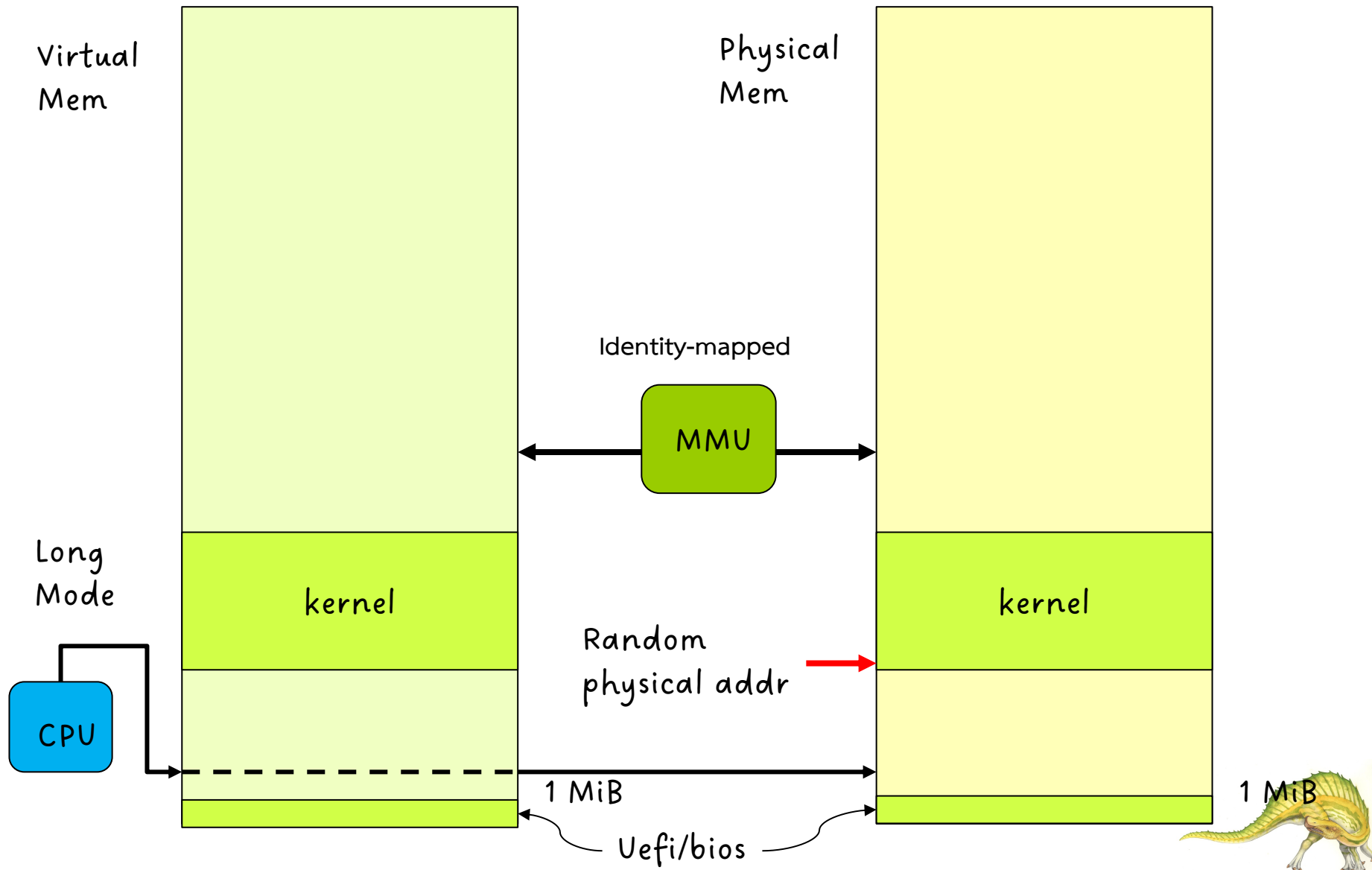
ขั้นตอน 3. Kernel เริ่มประมวลผล

- UEFI จองพื้นที่ Memory และ โหลด compress kernel code
- UEFI รัน decompress stub เพื่อ decompress kernel code (vmlinuz)
- decompress stub ใช้ KASLR (Kernel Address Space Layout Randomization) เพื่อกำหนดค่า physical memory address แบบ random
- physical address ที่จะ decompress kernel มาไว้ใน memory อาจเป็น address ไหนก็ได้ที่ไม่ overlap กับ memory address ที่จองไว้เป็นพิเศษ เช่น ที่จองให้ devices หรือตารางต่างๆ (GDT, IDT, etc) ที่ UEFI ทำให้





Kernel uses identity-mapped





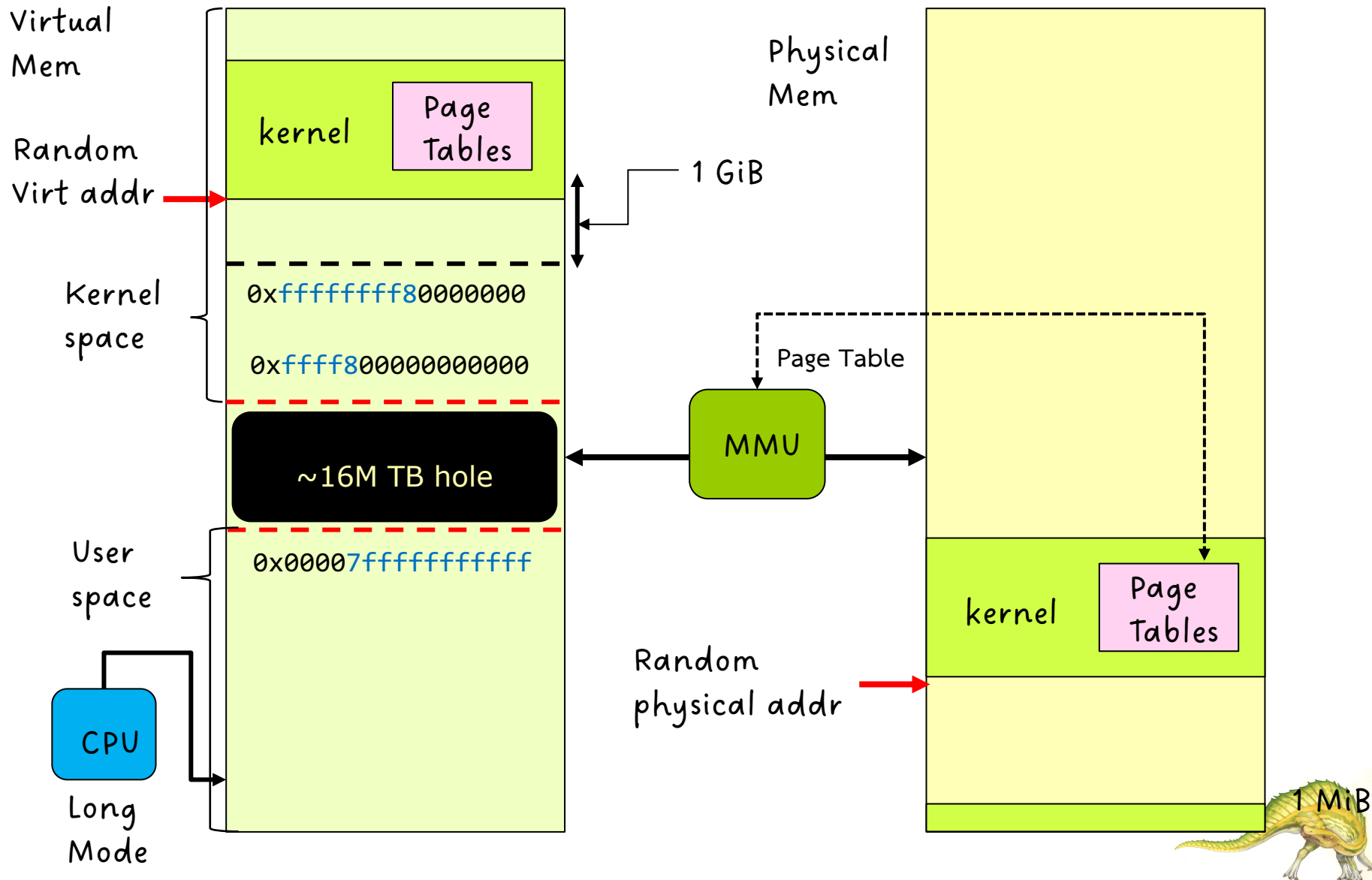
ขั้นตอน 4. Kernel เปลี่ยนไปใช้ Page Table

- ซีพียูประมวลผล Linux Kernel และสร้างตารางสำคัญเช่น GDT IDT และ drivers ของมันเอง แล้วป้องกันการเข้าถึงโค้ดและข้อมูลของ UEFI (เนื่องจากมันไม่ไว้ใจ UEFI – UEFI เป็นเหมือนคนส่งของที่เสร็จงานแล้วต้องแยกออกไป)
 - ซีพียูยังเข้าถึง Memory แบบ Identity-mapped อยู่
- Kernel สร้าง Page Table สำหรับตัวมันเอง
- Kernel เปลี่ยนการเข้าถึง Memory ของซีพียูจาก identity-mapped เป็น **แบบใช้ Page Table** การเปลี่ยนครั้งนี้จะทำให้ Kernel เปลี่ยนไปอยู่ใน Memory address สูง `0xffffffff80000000`
- Kernel ใช้ KASLR (Kernel Address Space Layout Randomization) อีกครั้ง เพื่อกำหนดค่า virtual address เริ่มต้น ให้เป็นค่า Random ที่อยู่ใน 1 GiB แรก





Kernel uses Page Table





ขั้นตอน 5. Kernel ประมวลผล

- Kernel กำหนดค่า PA ของ Page Table ของ Kernel ใน CR3 register
- Kernel ใช้วิธีการ “long jump” เพื่อประมวลผลต่อโดยอ้างอิงชุดคำสั่งด้วย High memory address
 - MMU จะใช้ Page Table (ที่ชี้โดย CR3) แปลงค่า VA เป็น PA





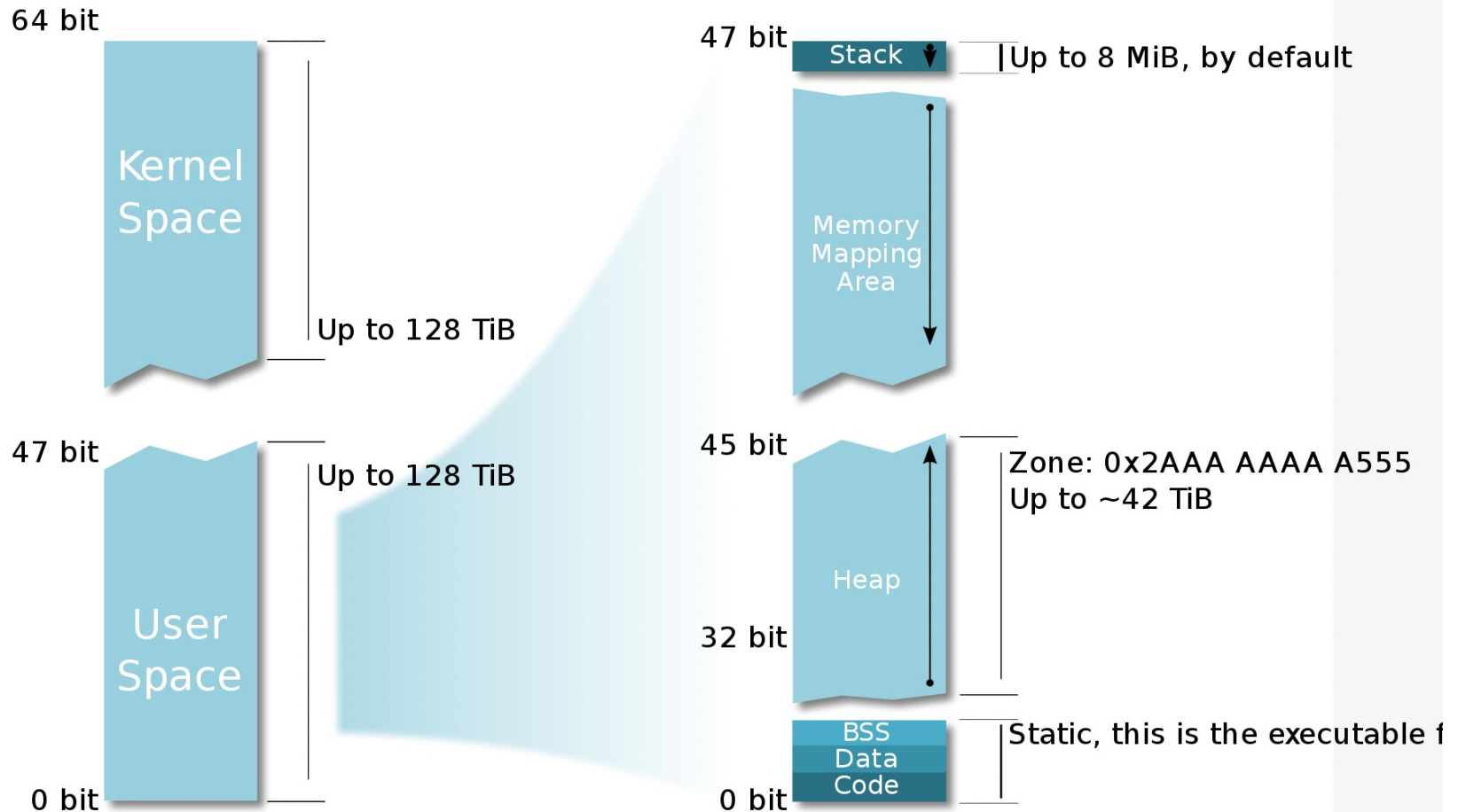
Linux Virtual Memory Layout

- เมื่อเริ่มต้นการทำงาน Linux จะ load kernel เข้าสู่ main memory ใน high logical memory address และจองพื้นที่นั้นไว้ใช้ตลอดเวลา
- Kernel's logical memory ตั้งแต่ `ffff800000000000` ถึง `ffffffffffffffff`
- User's logical memory ตั้งแต่ `0000000000000000` ถึง `00007fffffffffff`
- OS สร้างและจัดการ page tables ใน kernel memory space





Linux Memory Layout (64 bits)



So Kernel + User Spaces add for 256 TiB which is a tiny part of the 16 777 216 TiB addressable over 64 bit!

https://en.wikibooks.org/wiki/The_Linux_Kernel/Memory#:~:text=Linux%20kernels%20split%20the%204GB,final%201GB%20starting%20at%200xc0000000





Linux Memory Layout (64 bits)

Start addr	Offset	End addr	Size	VM area description
0000000000000000	0	00007fffffffffff	128 TB	user-space virtual memory, different per mm
0000800000000000	+128 TB	ffff7fffffffffff	~16M TB	... huge, almost 64 bits wide hole of non-canonical virtual memory addresses up to the -128 TB starting offset of kernel mappings.
				Kernel-space virtual memory, shared between all process
ffff800000000000	-128 TB	ffff87ffffffffff	8 TB	... guard hole, also reserved for hypervisor
ffff880000000000	-120 TB	ffff887ffffffffff	0.5 TB	LDT remap for PTI
ffff888000000000	-119.5 TB	ffffc87ffffffffff	64 TB	direct mapping of all physical memory (page_offset_base)
ffffc88000000000	-55.5 TB	ffffc87ffffffffff	0.5 TB	... unused hole
ffffc90000000000	-55 TB	ffffe87ffffffffff	32 TB	vmalloc/ioremap space (vmalloc_base)
ffffe90000000000	-23 TB	ffffe97ffffffffff	1 TB	... unused hole
ffffea0000000000	-22 TB	ffffeaffffffffffff	1 TB	virtual memory map (vmemmap_base)
fffffeb000000000	-21 TB	fffffeb7ffffffffff	1 TB	... unused hole
fffffec000000000	-20 TB	fffffb7ffffffffff	16 TB	KASAN shadow memory





Identical layout to the 56-bit one from here on:

fffffc0000000000	-4	TB	fffffdffffffffffff	2 TB	... unused hole
fffffe0000000000	-2	TB	fffffe7ffffffffffff	0.5 TB	vaddr_end for KASLR
fffffe8000000000	-1.5	TB	fffffeffffffffffff	0.5 TB	cpu_entry_area mapping
ffffff0000000000	-1	TB	ffffff7ffffffffffff	0.5 TB	... unused hole
ffffff8000000000	-512	GB	ffffffeefffffffffffff	444 GB	%esp fixup stacks
ffffffef00000000	-68	GB	ffffffeefffffffffffff	64 GB	... unused hole
fffffff000000000	-4	GB	fffffff7ffffffffffff	2 GB	EFI region mapping space
<u>fffffff800000000</u>	-2	GB	fffffff9ffffffffffff	512 MB	... unused hole
fffffff800000000	-2048	MB			<u>kernel text mapping, mapped to physical address 0</u>
fffffffaf0000000	-1536	MB	fffffffefffffffffffff	1520 MB	module mapping space
fffffff0000000	-16	MB			
FIXADDR_START	~-11	MB	fffffff5ffffffffffff	~0.5 MB	kernel-internal fixmap range, variable size and offset
fffffff6000000	-10	MB	fffffff600ffff	4 kB	legacy vsyscall ABI
fffffffefe000000	-2	MB	fffffffefeffffff	2 MB	... unused hole





โครงสร้าง Page Table

- **แบบดั้งเดิม: Traditional** ทุก Process จะมี logical memory space ทั้งหมด (รวมทั้งส่วนของ Kernel) แต่ user process จะเข้าถึงได้ใน address ที่เป็น user space เท่านั้น
- **แบบใหม่: Kernel Page Table Isolation (KPTI)** แยก user page table กับ kernel page table ออกจากกัน เพื่อป้องกันการโจมตี (จาก meltdown attack)





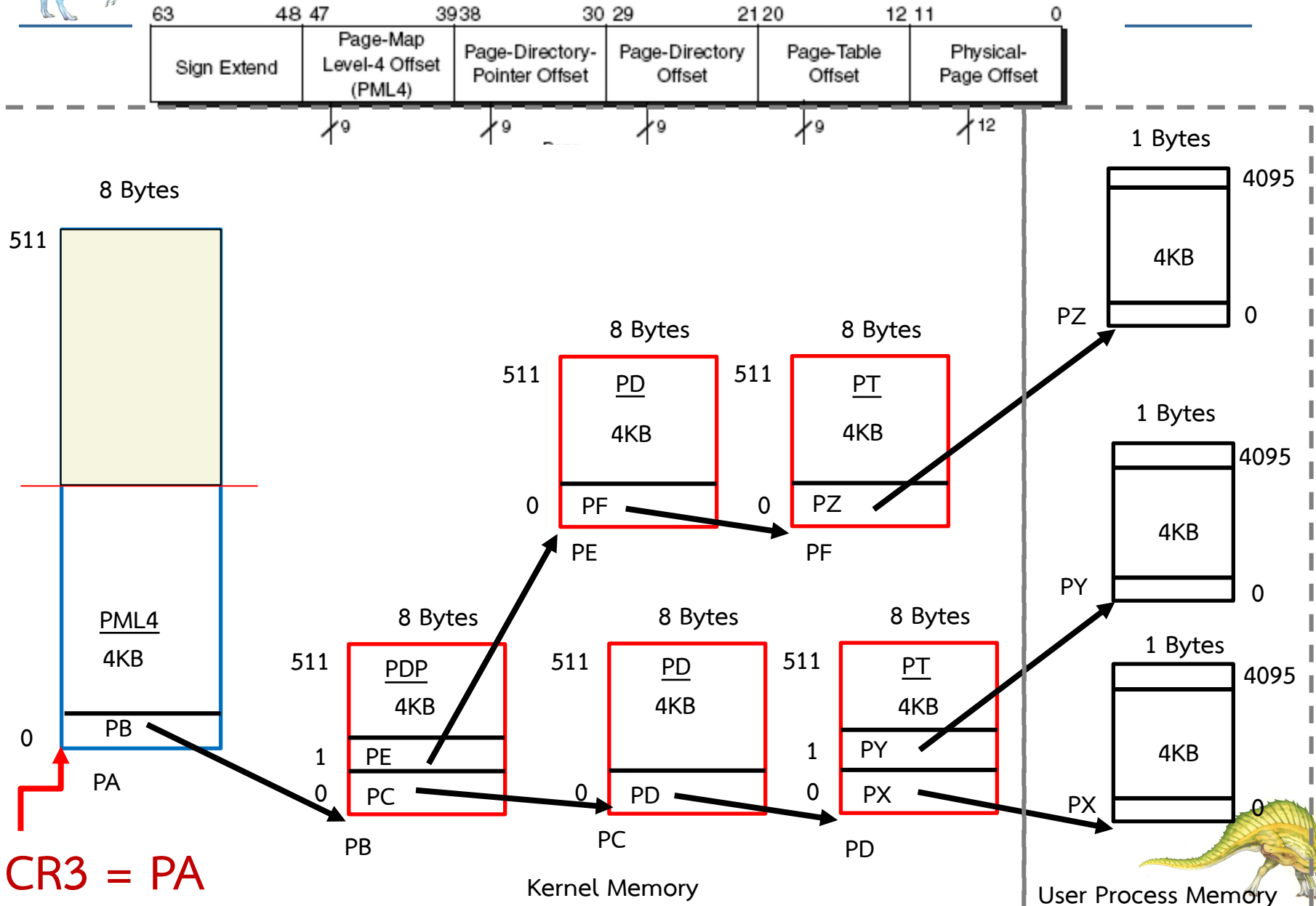
โครงสร้าง Page Table แบบดั้งเดิม

- Page Table ของทุก Process จะมี ส่วนที่รวมเนื้อหาของ kernel อยู่ด้วย ใน ส่วนที่ cover high memory address
- เนื้อหาของส่วน kernel ในทุก process จะเหมือนกัน
- เมื่อ process ประมวล OS จะกำหนดค่า cr3
 - เมื่อ process ใช้ system call ไม่ต้องเปลี่ยน cr3
 - เมื่อเกิด interrupt ในระหว่างการประมวลผล process ไม่ต้องเปลี่ยน cr3
 - kernel เปลี่ยน cr3 เมื่อ scheduler เลือก process ใหม่เข้าใช้ cpu
- ในระหว่างที่ kernel ประมวลผล มันจะสามารถเข้าถึง memory ใน user space ได้



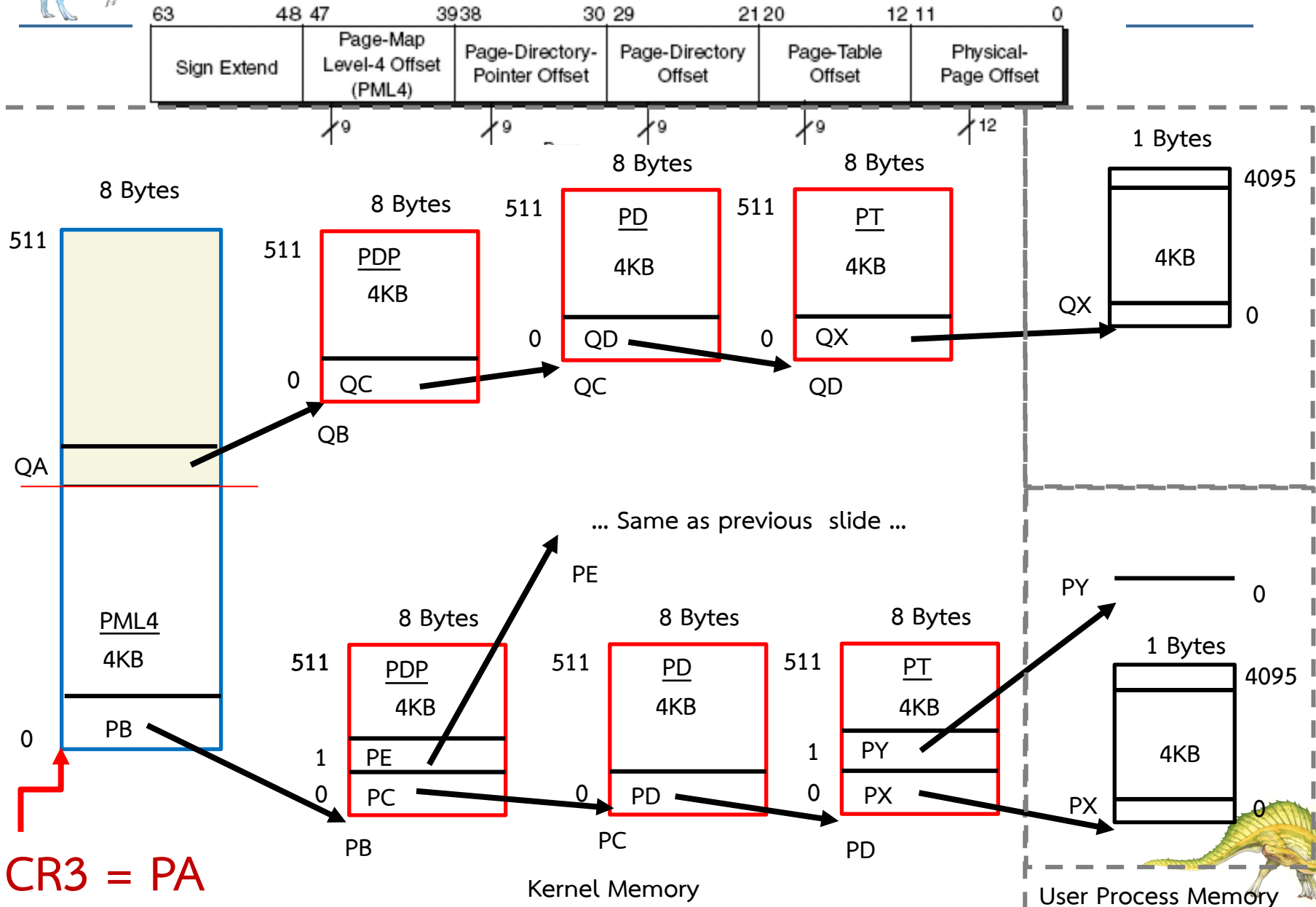


Page Table Structure in User space





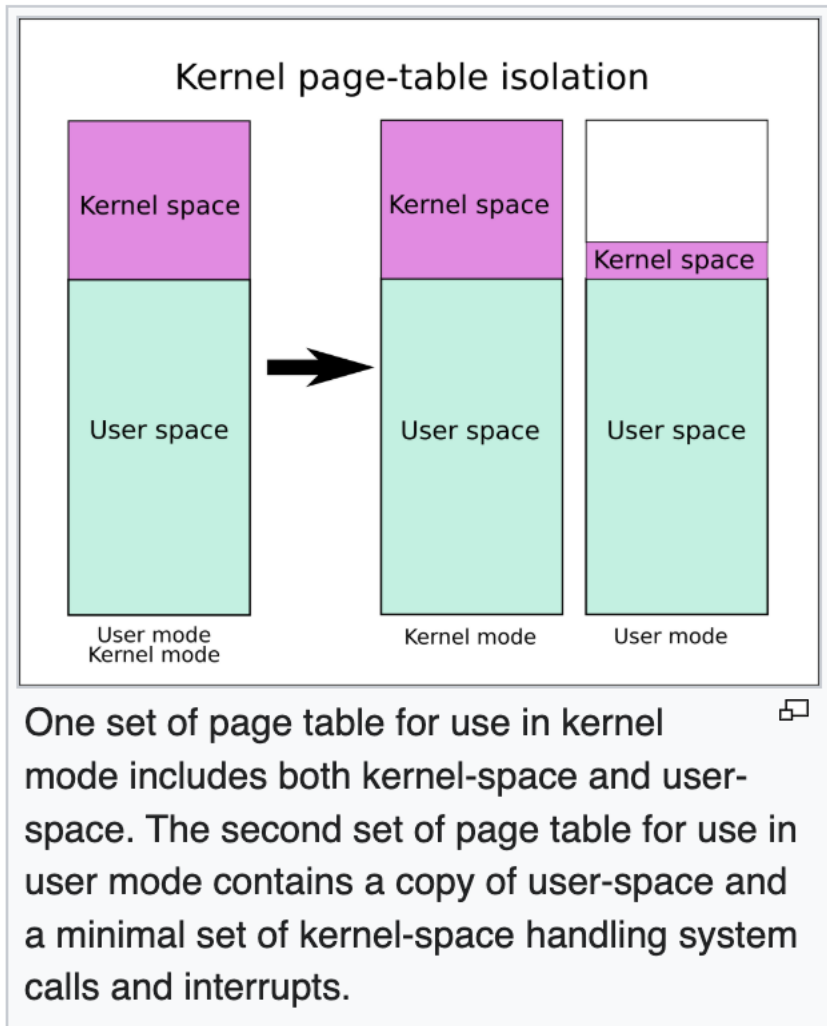
Page Table Structure in User space & Kernel space





Kernel Page Table Isolation (KPTI)

Kernel page-table isolation (KPTI or PTI,^[1] previously called **KAISER)**^{[2][3]} is a [Linux kernel](#) feature that mitigates the [Meltdown security vulnerability](#) (affecting mainly [Intel's x86 CPUs](#))^[4] and improves kernel hardening against attempts to bypass [kernel address space layout randomization \(KASLR\)](#). It works by better isolating [user space](#) and kernel space memory.^{[5][6]} KPTI was merged into Linux kernel version 4.15,^[7] and [backported](#) to Linux kernels 4.14.11, 4.9.75, and 4.4.110.^{[8][9][10]} [Windows](#)^[11] and [macOS](#)^[12] released similar updates. KPTI does not address the related [Spectre](#) vulnerability.^[13]



https://en.wikipedia.org/wiki/Kernel_page-table_isolation





โครงสร้าง Page Table แบบ KPTI

- Kernel Page Table Isolation (KPTI) เป็นการแยก Page Table (PT) ของ kernel ออกจาก Page Table ของ Process
- ในกรณี process ใช้ system call :
 1. OS จะต้อง copy ค่าการ mapping จาก VA ไป PA ใน PT ของ user process ที่ kernel จำเป็นต้องใช้เพื่อเข้าถึงข้อมูลใน User space memory มายัง kernel PT เพื่อให้ kernel สามารถเข้าถึง user memory ได้ (เท่าที่จำเป็นเท่านั้น)
 2. OS จะต้องเปลี่ยน cr3 ให้ชี้ไปที่ Kernel Page Table และ
 3. OS เปลี่ยน cr3 กลับเป็นของ process เมื่อ system call return





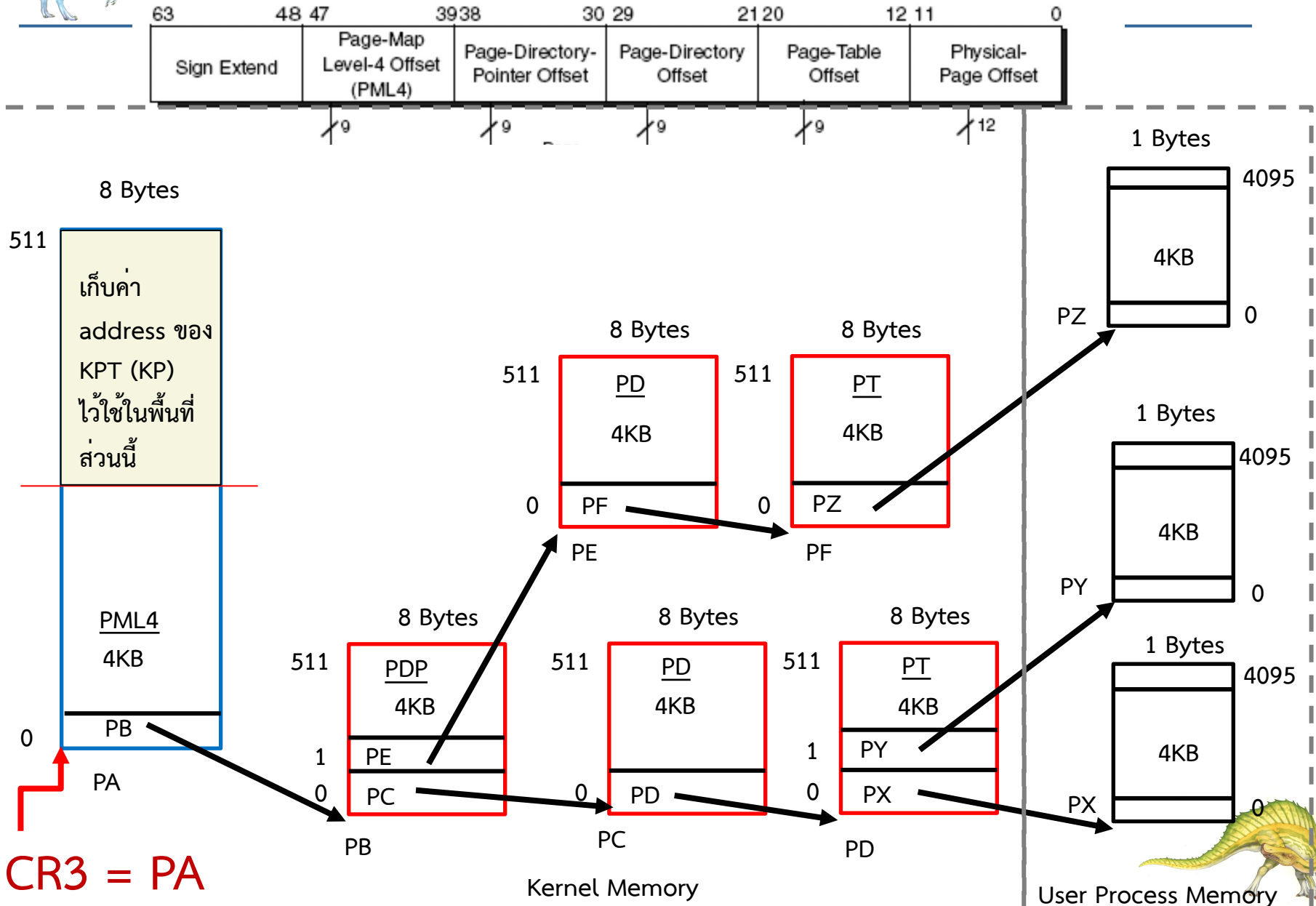
โครงสร้าง Page Table แบบ KPTI

- ในกรณีที่เกิด interrupt ในระหว่างการประมวลผล process ใดๆ
 1. OS ต้องเปลี่ยน cr3 ให้ชี้ไปที่ Kernel Page Table เพื่อประมวลผล Interrupt handler และ
 2. เปลี่ยน cr3 กลับเป็นของ process ที่ถูกขัดจังหวะเมื่อ resume การทำงานของ Process นั้น



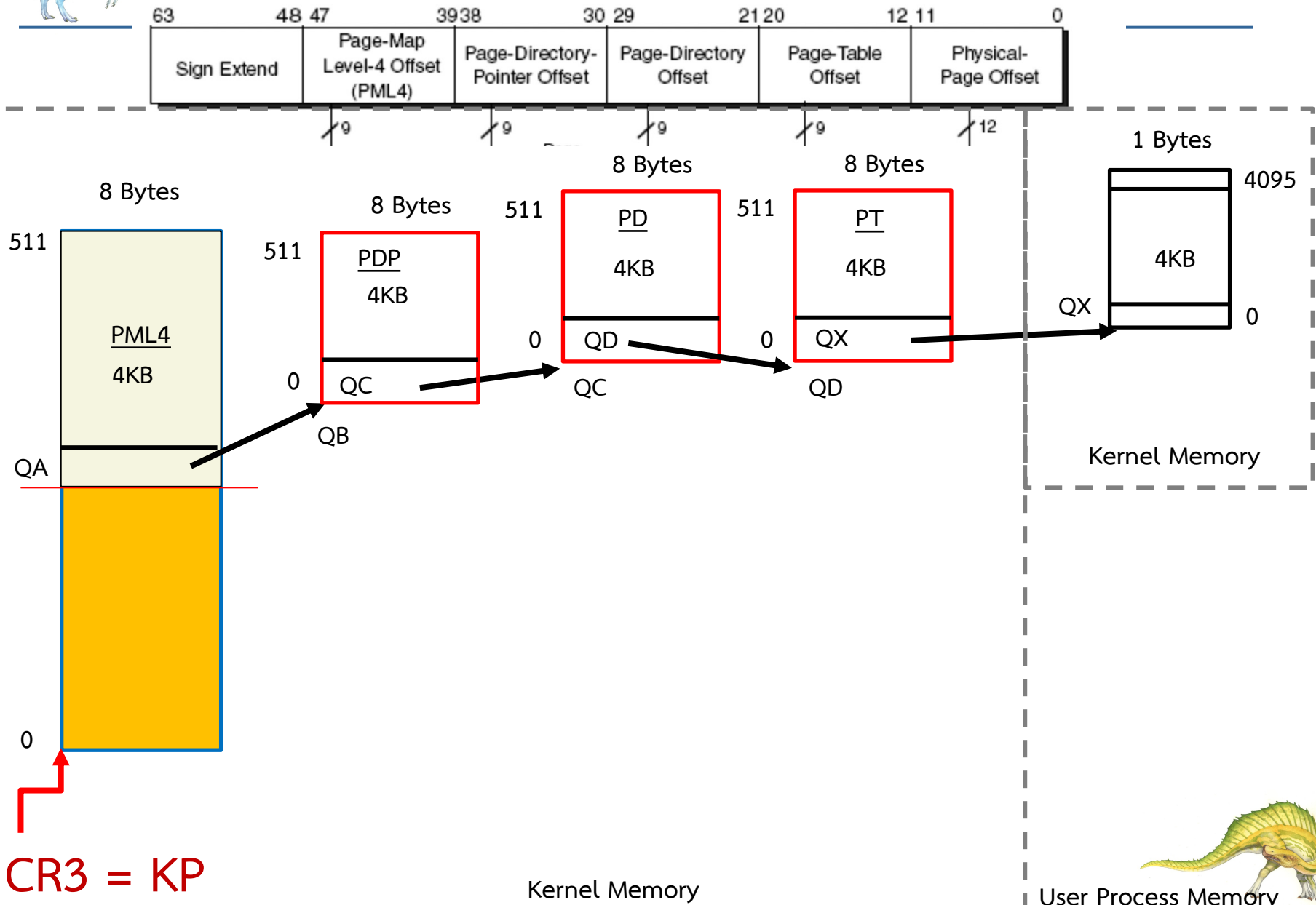


Page Table Structure in User space



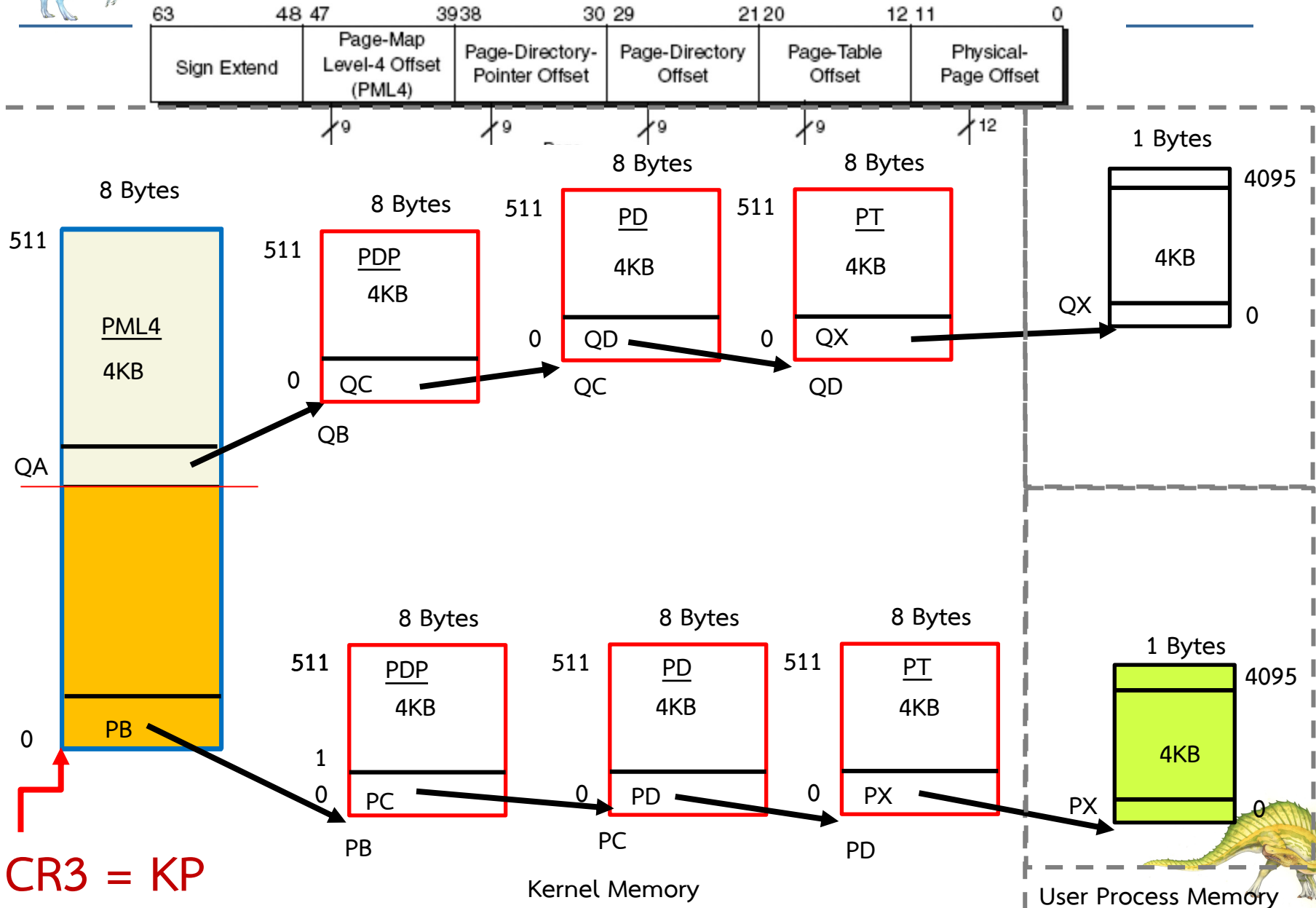


เปลี่ยน CR3 ชี้ Page Table Structure in Kernel space





Copy เฉพาะ Mapping ที่ต้องใช้มายัง Kernel space





การเขียน **Program Dynamic Memory Allocation**





Dynamic Memory Allocation

Dynamic Memory Allocation in C

Last Updated : 9 Apr, 2026



Dynamic memory allocation allows a programmer to allocate, resize, and free memory at runtime. Key advantages include.

- Memory is allocated on the heap area instead of stack. Please refer [memory layout of C programs](#) for details
 - Array size can be increased or decreased as needed.
 - Memory persists even after the function that allocated it finishes, allowing functions to return pointers to it. This is different from stack allocated variables as it is not safe to return address of those variable.
-
- เนื้อหานำมาจาก <https://www.geeksforgeeks.org/c/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc/>





Dynamic Memory Allocation

- Dynamic Memory Allocation คือการจัดสรรพื้นที่เป็น block เพื่อจัดเก็บข้อมูล ใน Heap segment ของ Process Memory Space
- คุณสมบัติ:
 1. ถ้าสำเร็จจะได้เป็น block สำหรับเก็บข้อมูลใน Heap
 2. ฟังก์ชันที่ใช้จะ return ค่าเป็น memory address ของพื้นที่เก็บข้อมูล
 3. ผู้ใช้สามารถเพิ่มหรือลดขนาดของพื้นที่นี้ได้
 4. พื้นที่ block สำหรับเก็บข้อมูลจะยังคงอยู่ถึงแม้ว่าคำสั่งที่ใช้จัดสรรพื้นที่หรือฟังก์ชันที่ออกคำสั่งดังกล่าวจะจบไปแล้ว
 5. ผู้ใช้ต้องใช้คำสั่ง free เพื่อปลดปล่อยพื้นที่ที่ถูกจัดสรรกลับไปให้ Heap
- เนื้อหานำมาจาก <https://www.geeksforgeeks.org/c/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc/>





malloc()

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main() {
    int *ptr = (int *)malloc(20);
```

```
    // Populate the array
    for (int i = 0; i < 5; i++)
        ptr[i] = i + 1;
```

```
    // Print the array
    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);
    return 0;
```

```
}
```

- malloc() จัดสรรพื้นที่ N=20 bytes จาก Heap Memory
- ไม่ clear ค่าให้

- เนื้อหานำมาจาก <https://www.geeksforgeeks.org/c/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc/>





malloc()

```
#include <stdio.h>
#include <stdlib.h>
```

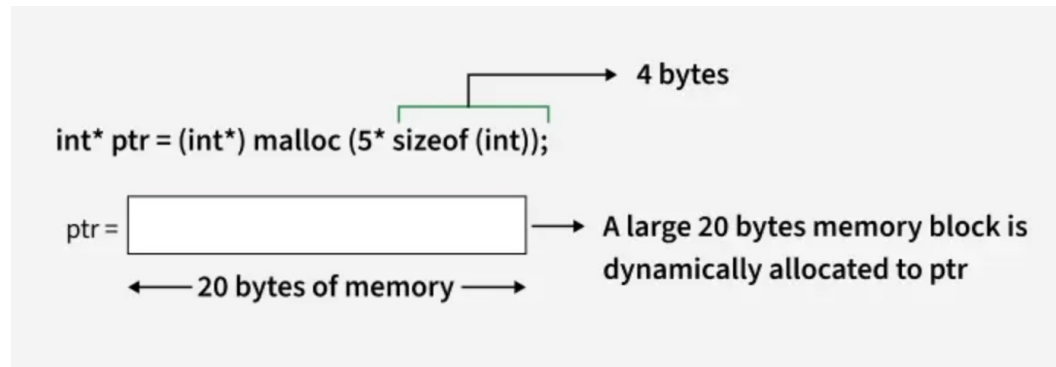
```
int main() {
    int *ptr = (int *)malloc(sizeof(int) * 5);
```

```
    // Checking if failed or pass
    if (ptr == NULL) {
        printf("Allocation Failed");
        exit(0);
    }
```

```
    // Populate the array
    for (int i = 0; i < 5; i++)
        ptr[i] = i + 1;
```

```
    // Print the array
    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);
    return 0;
}
```

malloc() จัดสรรพื้นที่ 5x(4)
bytes จาก Heap Memory



- เนื้อหา <https://www.geeksforgeeks.org/c/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc>





calloc()

```
#include <stdio.h>
#include <stdlib.h>
```

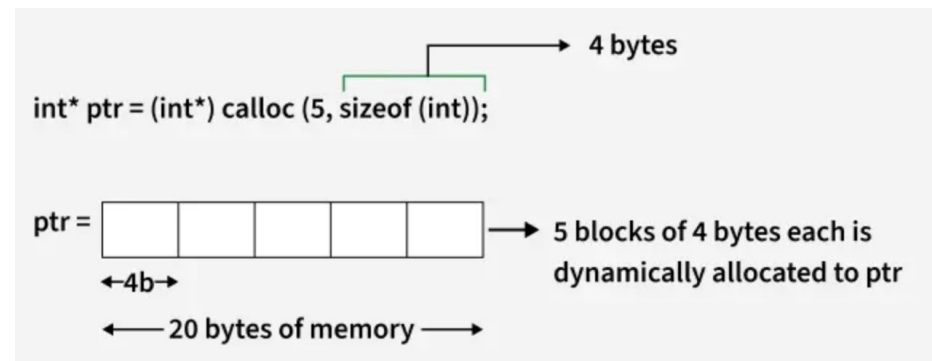
```
int main() {
    int *ptr = (int *)calloc(5, sizeof(int));
```

```
    // Checking if failed or pass
    if (ptr == NULL) {
        printf("Allocation Failed");
        exit(0);
    }
```

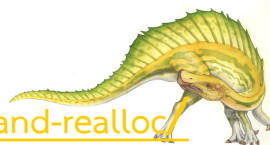
```
    // No need to populate as already
    // initialized to 0
```

```
    // Print the array
    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);
    return 0;
}
```

- calloc() จัดสรรพื้นที่ $N=5 \times (4)$ bytes จาก Heap Memory
- clear ค่าให้ (assign 0)



- เนื้อหา <https://www.geeksforgeeks.org/c/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc>





free()

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main() {
    int *ptr = (int *)calloc(5, sizeof(int));
```

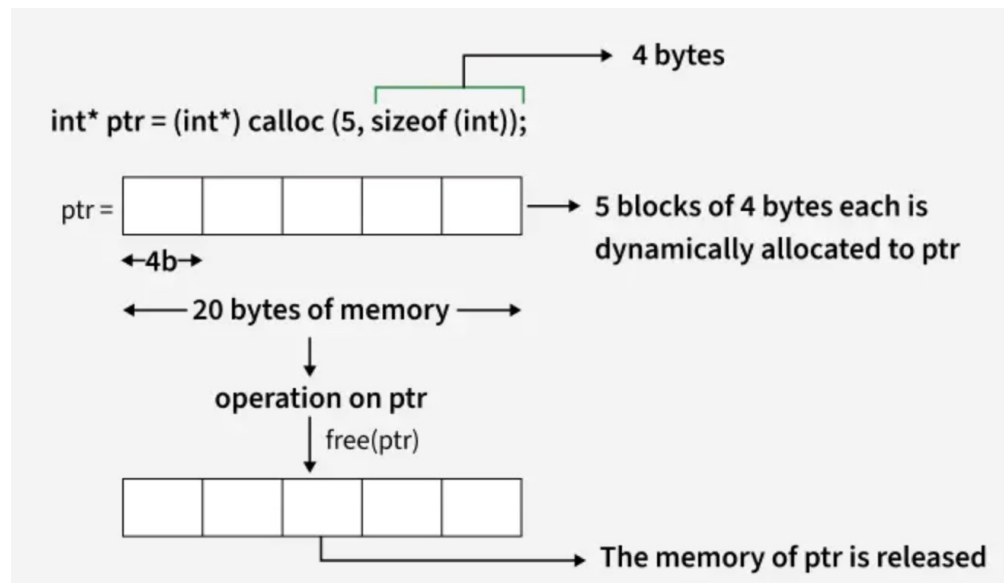
```
    // Do some operations.....
    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);
```

```
    free(ptr);
```

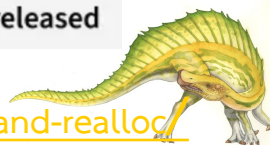
```
    return 0;
```

```
}
```

- free() คือพื้นที่ memory ที่จัดสรรมาให้กับ Heap
 - C ไม่มี garbage collector เหมือน java และ go lang
 - ถ้าไม่ได้ free() อาจทำให้เกิด memory leak



- เนื้อหา <https://www.geeksforgeeks.org/c/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc>





realloc()

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main()
{
```

```
    int *ptr = (int *)malloc(5 * sizeof(int));
```

```
    // Reallocation
```

```
    int *temp = (int *)realloc(ptr, 10 * sizeof(int));
```

```
    // Only update the pointer if reallocation is successful
```

```
    if (temp == NULL)
```

```
        printf("Memory Reallocation Failed\n");
```

```
    else{
```

```
        if (ptr == temp) printf("old block\n");
```

```
        else printf("new block\n");
```

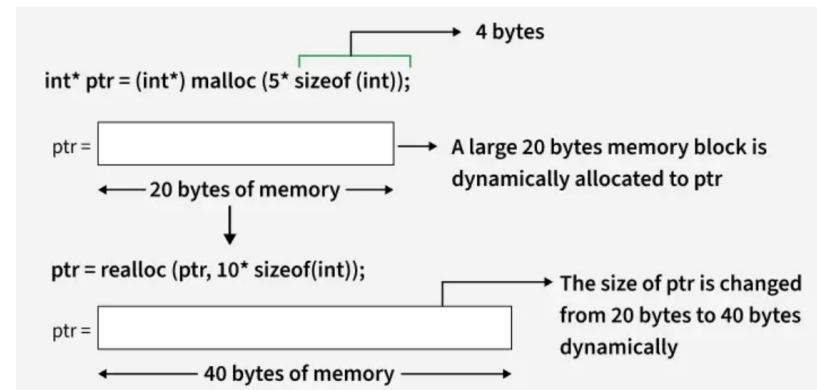
```
        ptr = temp;
```

```
    }
```

```
    return 0;
```

```
}
```

- realloc() คือฟังก์ชันสำหรับขยาย/ลดพื้นที่ memory ที่ใช้เก็บข้อมูลใน Heap segment



- เนื้อหา <https://www.geeksforgeeks.org/c/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc>





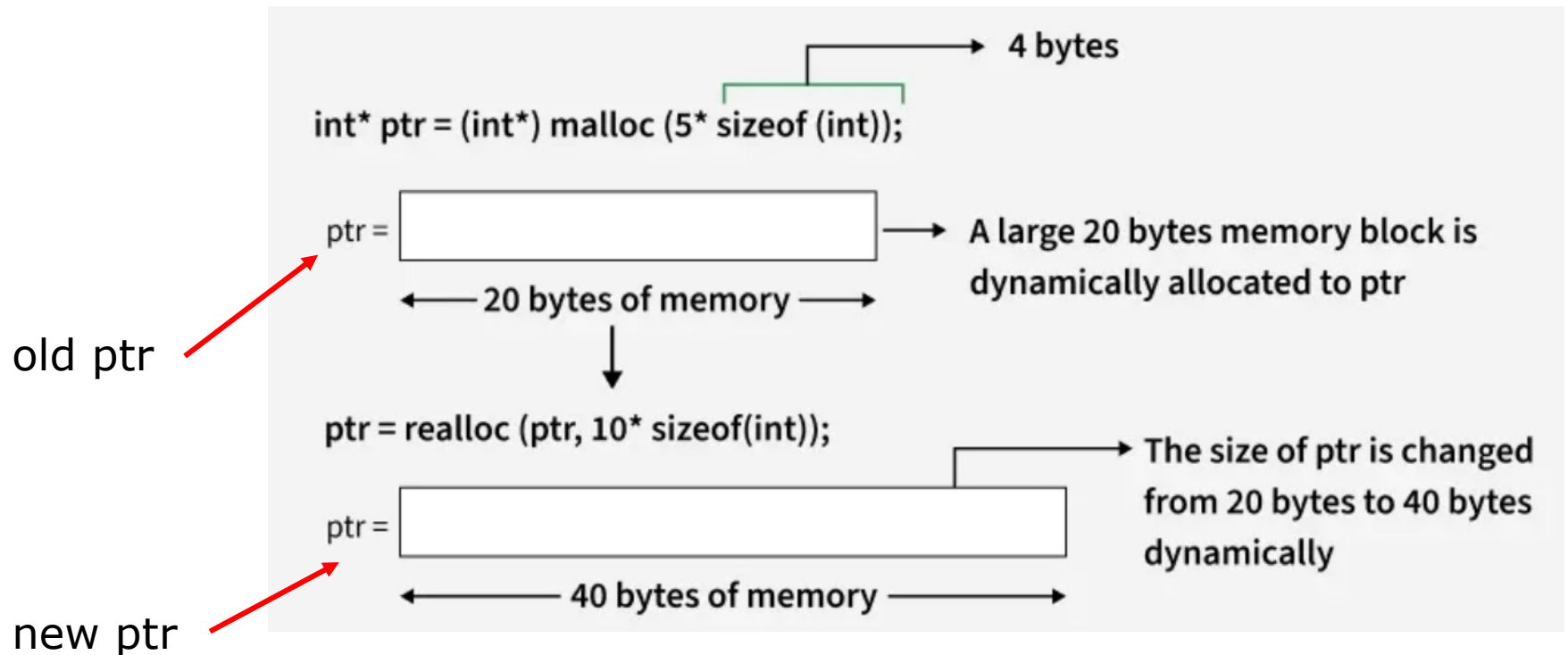
realloc()

- ถ้าใช้คำสั่ง realloc() เพื่อขยายพื้นที่ แล้วพื้นที่ block ที่ติดกัน ที่ชี้โดย ptr (pointer เดิม) เพียงพอ realloc() จะ return ค่าของ ptr
- ถ้าใช้คำสั่ง realloc() เพื่อขยายพื้นที่ แล้วพื้นที่ block ที่ติดกัน ที่ชี้โดย ptr (pointer เดิม) ไม่เพียงพอ realloc() จะทำดังนี้
 1. malloc() พื้นที่ block ใหม่ที่มีพื้นที่ติดกันเพียงพอตามที่ขยาย
 2. Copy ข้อมูลจาก block เดิมไปยัง block ใหม่
 3. free พื้นที่ block เดิมที่ชี้โดย ptr (pointer เดิม)
 4. return ค่า memory address ของ block ใหม่
- ถ้าคำสั่ง realloc() fails มันจะ return NULL ในกรณีนี้ ptr จะยังคงชี้ไปที่ block เดิม ผู้ใช้ควรระวังอย่าลืม free(ptr)
- เนื้อหา <https://www.geeksforgeeks.org/c/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc/>



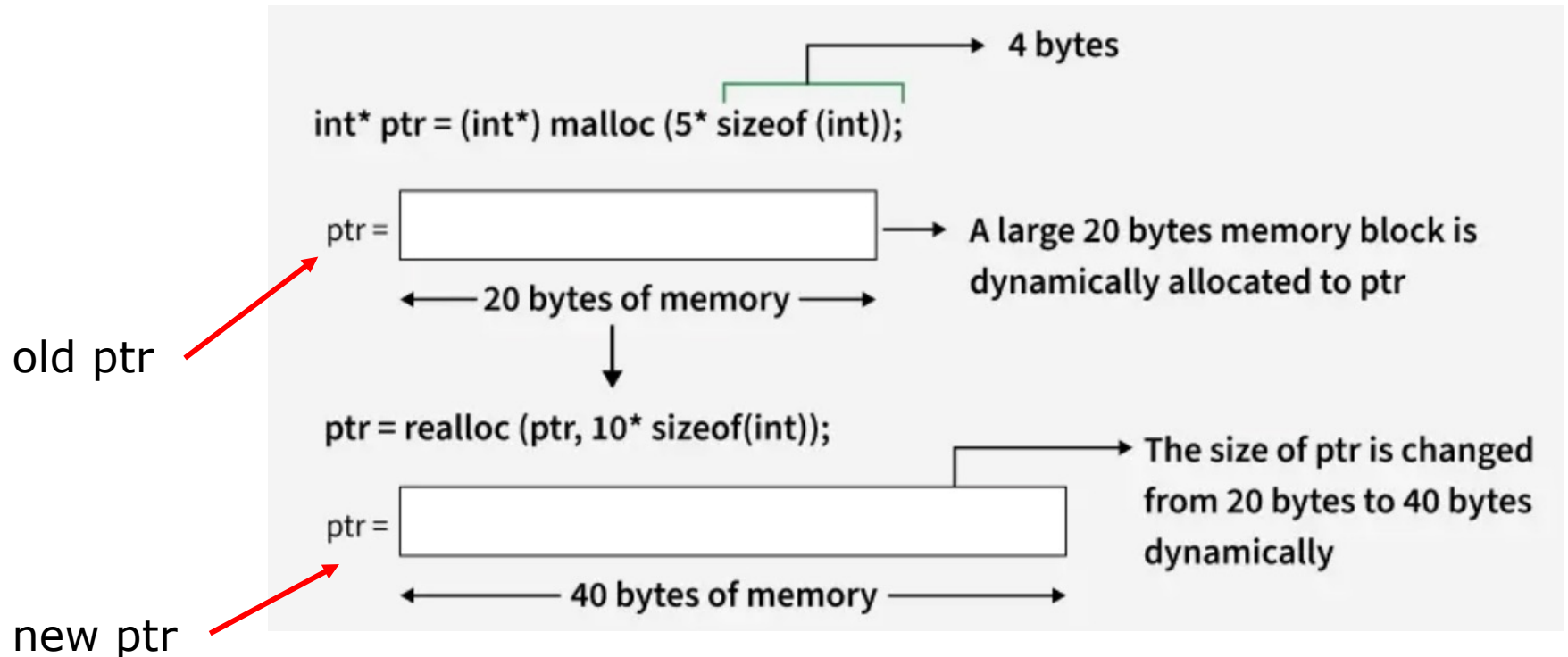


New ptr == Old ptr



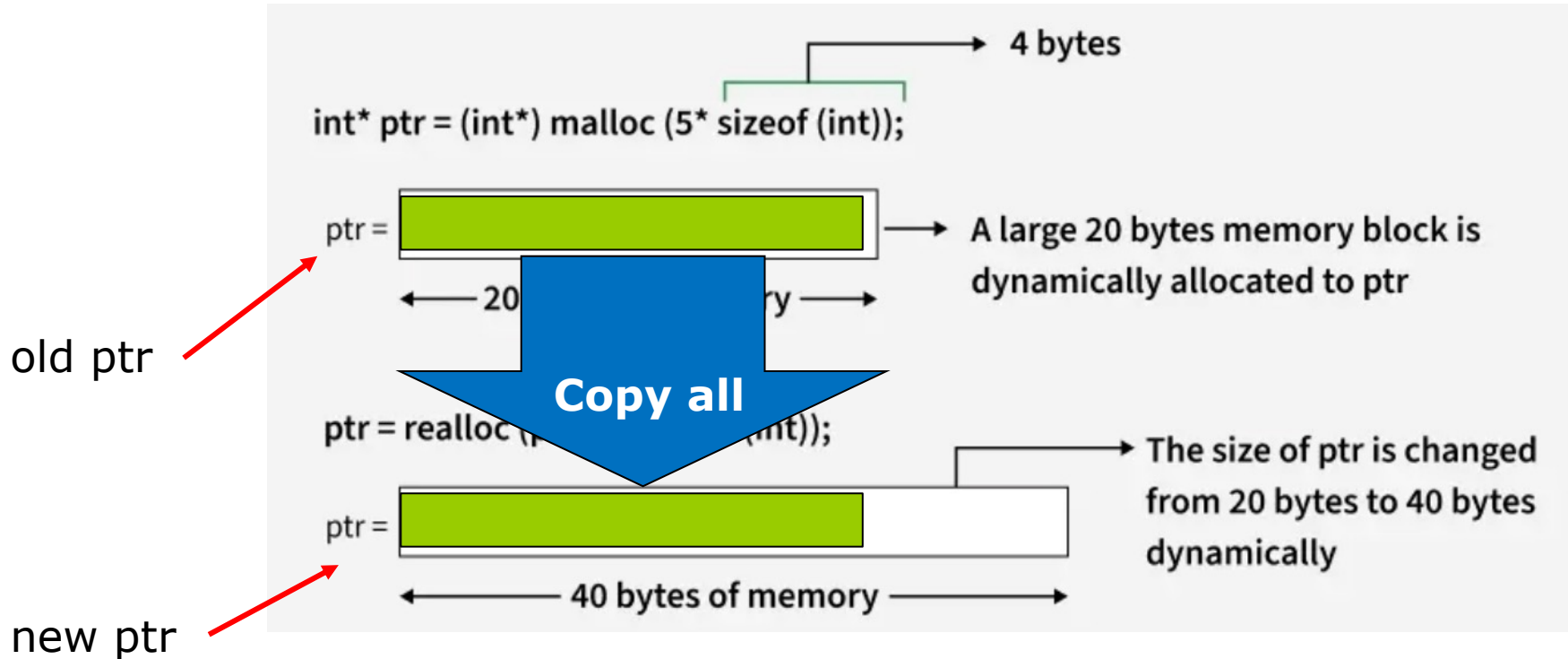


New ptr != Old ptr





New ptr != Old ptr

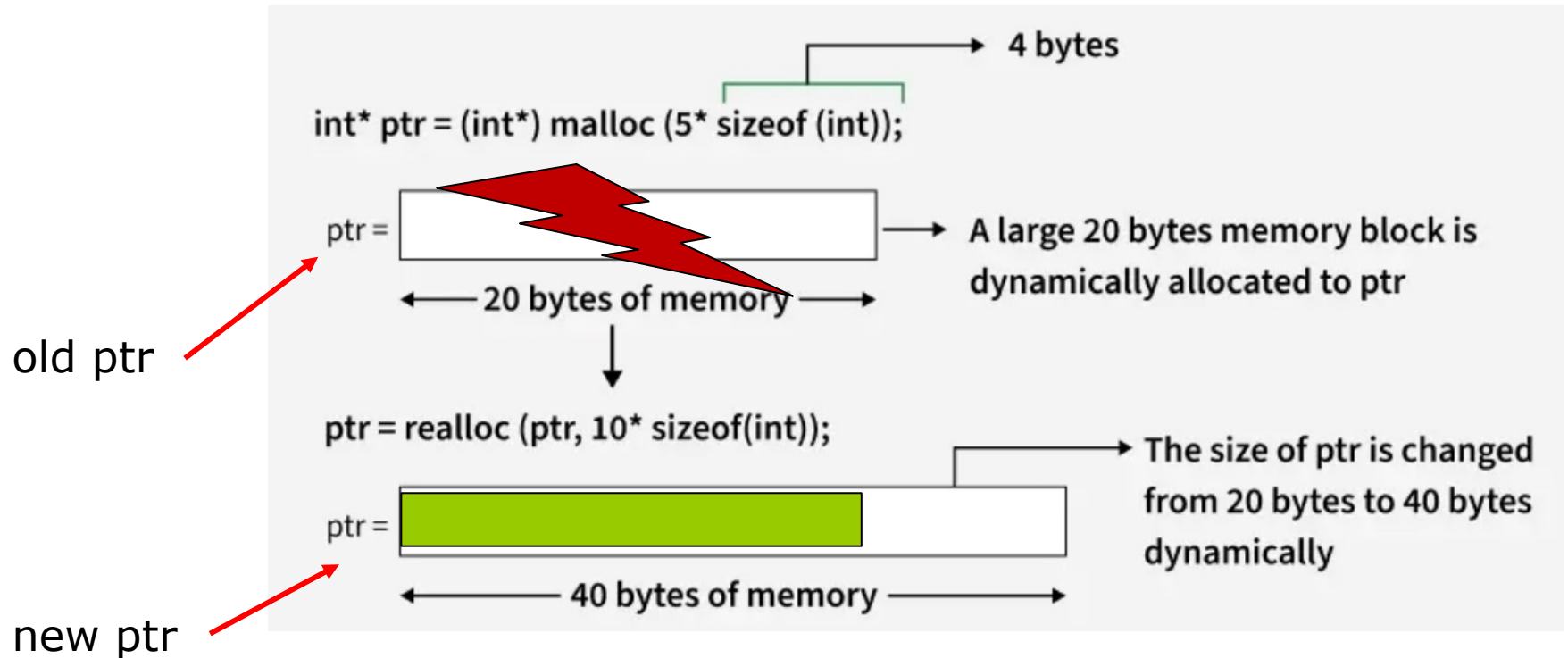


1. malloc() พื้นที่ block ใหม่ที่มีพื้นที่ติดกันเพียงพอตามที่ขยาย
2. Copy ข้อมูลจาก block เดิมไปยัง block ใหม่





New ptr != Old ptr



1. malloc() พื้นที่ block ใหม่ที่มีพื้นที่ติดกันเพียงพอตามที่ขยาย
2. Copy ข้อมูลจาก block เดิมไปยัง block ใหม่
3. free พื้นที่ block เดิมที่ชี้โดย ptr (pointer เดิม)
4. return ค่า memory address ของ block ใหม่





realloc() example

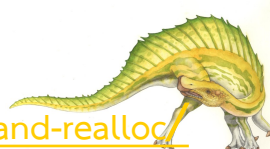
```
#include <stdio.h>
#include <stdlib.h>
int main() {

    int *ptr = (int *)malloc(5 * sizeof(int));
    if (ptr == NULL) {
        printf("Memory Allocation Failed\n");
        exit(0);
    }

    ptr = (int *)realloc(ptr, 8 * sizeof(int));
    if (ptr == NULL) {
        printf("Memory Reallocation Failed\n");
        exit(0);
    }

    // Assume we only use 5 elements now
    for (int i = 0; i < 5; i++) {
        ptr[i] = (i + 1) * 10;
    }
}
```

- เนื้อหา <https://www.geeksforgeeks.org/c/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc>





realloc() example

```
// Shrink the array back to 5 elements
ptr = (int *)realloc(ptr, 5 * sizeof(int));

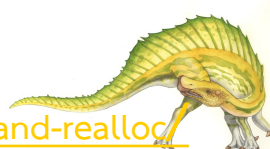
// Check if shrinking was successful
if (ptr == NULL) {
    printf("Memory Reallocation Failed\n");
    exit(0);
}

for (int i = 0; i < 5; i++)
    printf("%d ", ptr[i]);

// Finally, free the memory when done
free(ptr);

return 0;
}
```

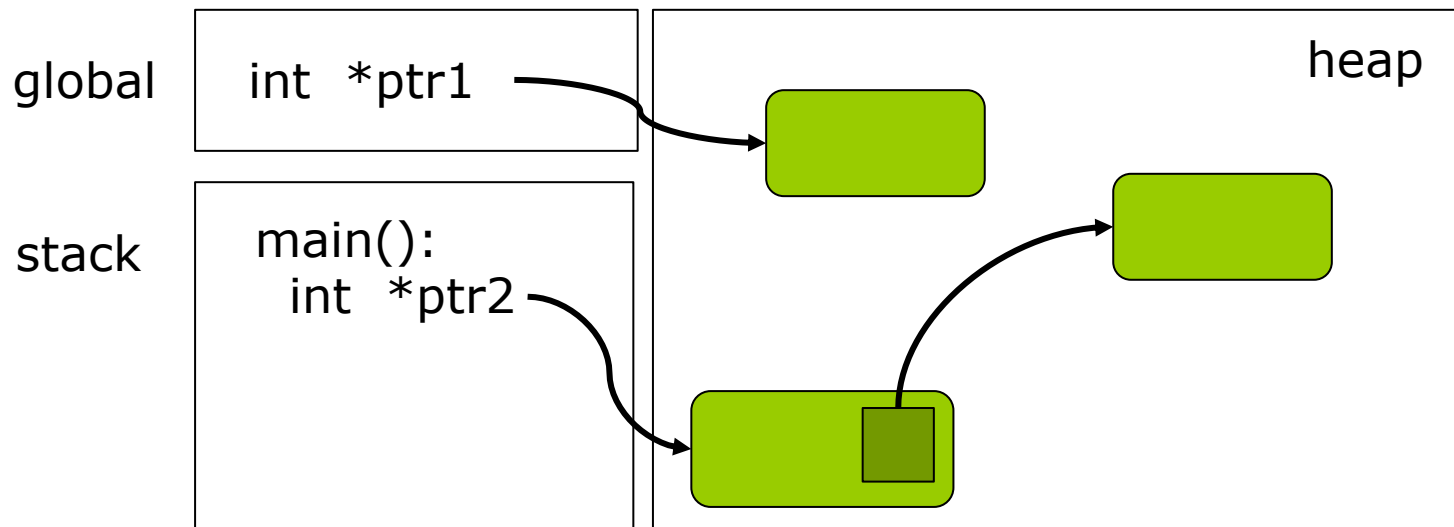
- เนื้อหา <https://www.geeksforgeeks.org/c/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc>





1. ปัญหา Memory Leak (1)

- Memory Leak: ถ้ามี memory block ที่ไม่มีตัวแปร pointer ใดชี้ไป พื้นที่นั้นก็เป็นพื้นที่ๆไม่มีใครรู้จัก และไม่สามารถ free ได้เพราะไม่มีตัวแปรชี้ว่ามันอยู่ที่ใด **ทำให้เสียพื้นที่ไปโดยเปล่าประโยชน์ เมื่อสะสมเยอะๆทำให้โปรแกรมช้า**



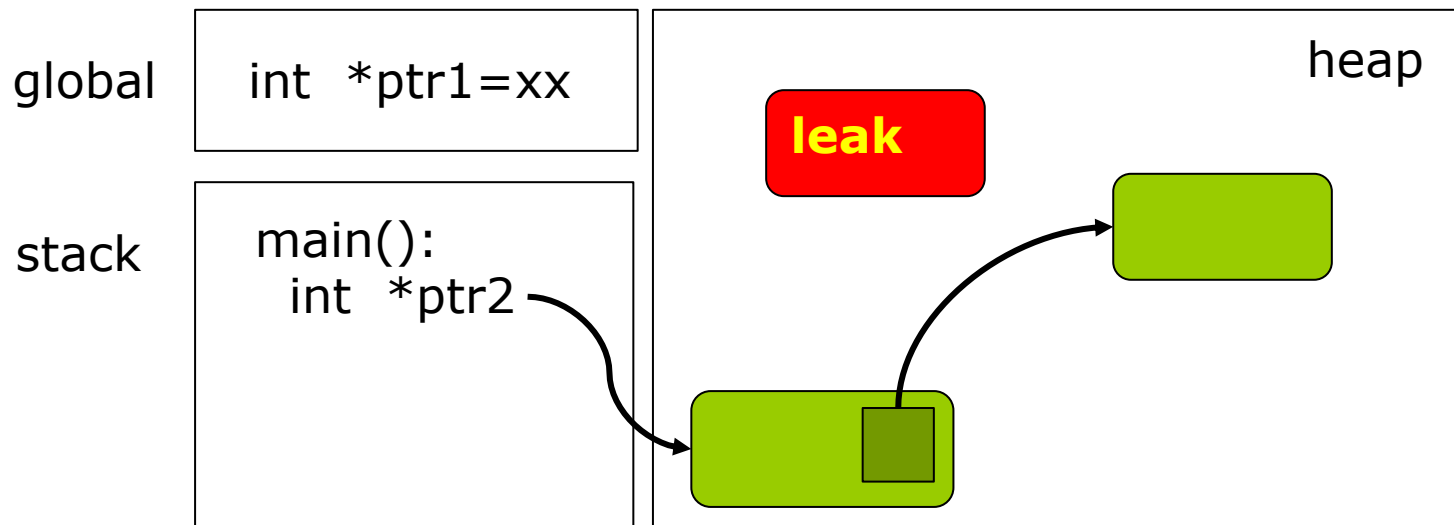
- เนื้อหา นำมาจาก <https://www.geeksforgeeks.org/c/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc/>





1. ปัญหา Memory Leak (2)

1. Memory Leak หลังจาก assign ค่า addr ใหม่หรือ NULL ให้กับ ptr1 ซึ่งเป็น **global var** ที่ชี้ไปยัง memory block



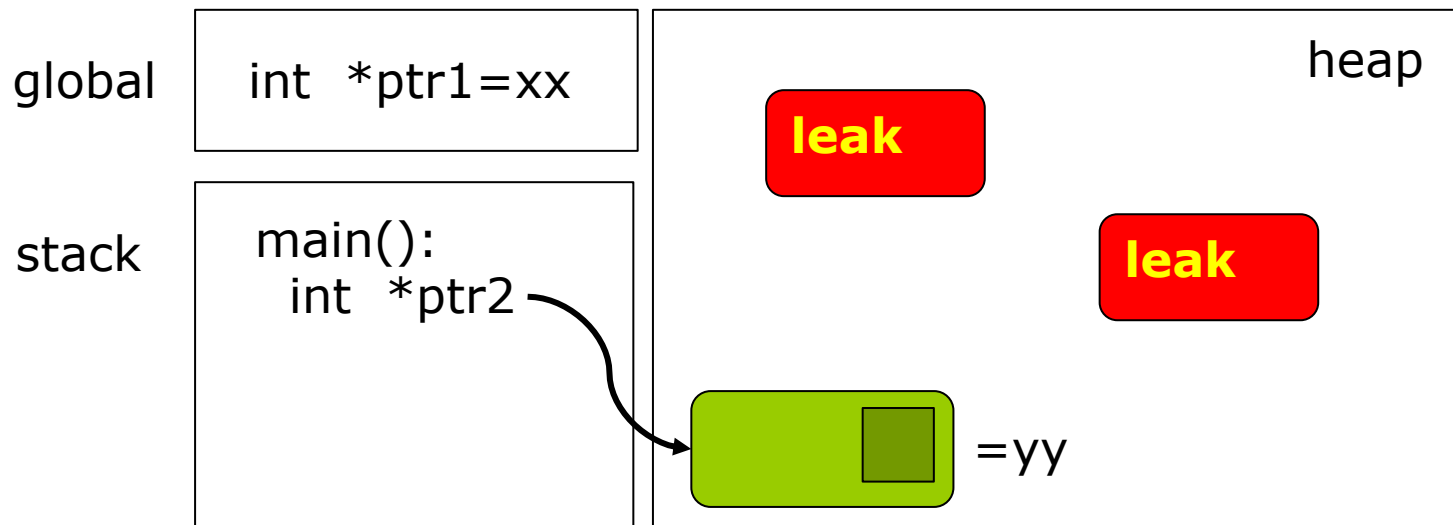
- เนื้อหานำมาจาก <https://www.geeksforgeeks.org/c/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc/>





1. ปัญหา Memory Leak

- Memory Leak หลังจาก assign ค่า addr ใหม่หรือ NULL ให้กับ **ptr** ที่อยู่ใน memory block ที่อยู่ใน **heap segment**



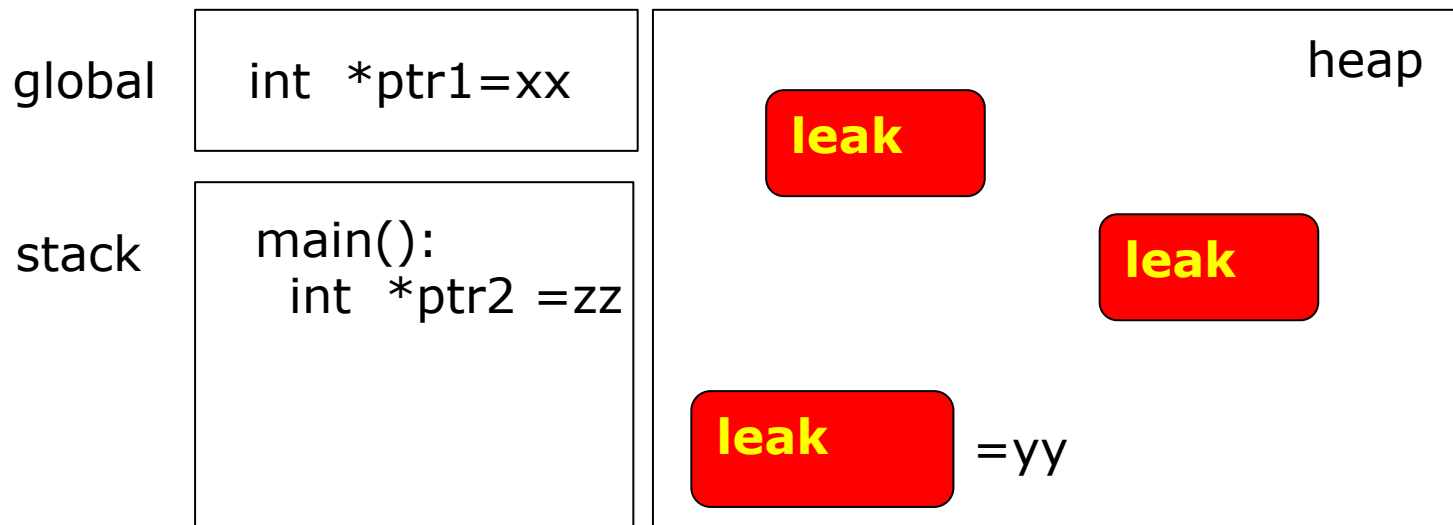
- เนื้อหา นำมาจาก <https://www.geeksforgeeks.org/c/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc/>





1. ปัญหา Memory Leak

- Memory Leak หลังจาก assign ค่า addr ใหม่หรือ NULL ให้กับ **ptr2** ที่อยู่ใน activation record ของ main() ใน **stack segment**



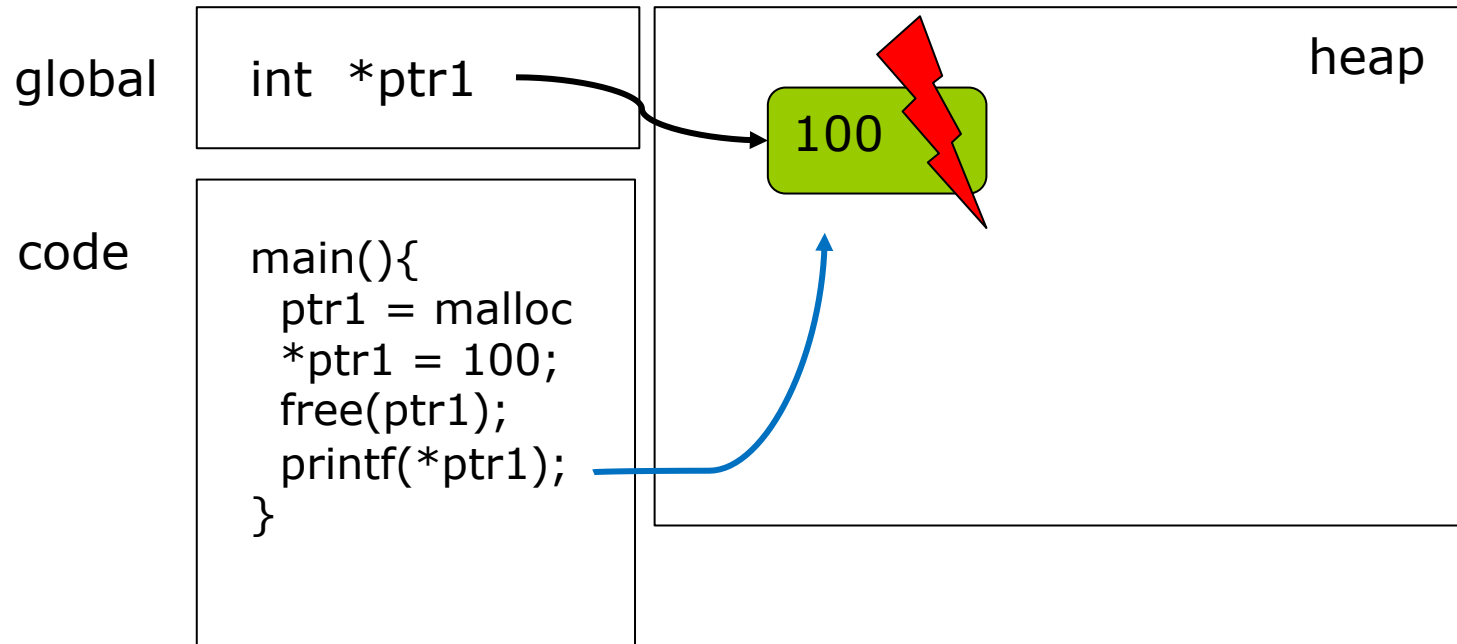
- เนื้อหานำมาจาก <https://www.geeksforgeeks.org/c/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc/>





2. ปัญหา Dangling Pointer

- การใช้ pointer หลังจาก free ไปแล้ว อาจทำให้เกิด undefined behavior หรือ โปรแกรมล่ม crash ได้



- เนื้อหา นำมาจาก <https://www.geeksforgeeks.org/c/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc/>

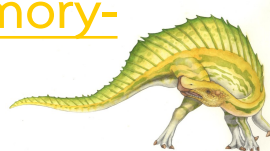




3. ปัญหา fragmentation

- ปัญหา Fragmentation คือในกรณีที่โปรแกรมมีการ allocate ข้อมูลชิ้นเล็กมากๆ ทำให้มีพื้นที่ติดกันน้อยลง ส่งผลให้เกิด external fragmentation ทำให้ malloc() ต้องขยายพื้นที่ของ heap segment ทั้งๆที่มีพื้นที่ว่างโดยรวมใน heap เดิมเพียงพอ แต่ไม่สามารถนำไปใช้ได้เนื่องจากไม่ใช่ memory block ที่ติดกัน

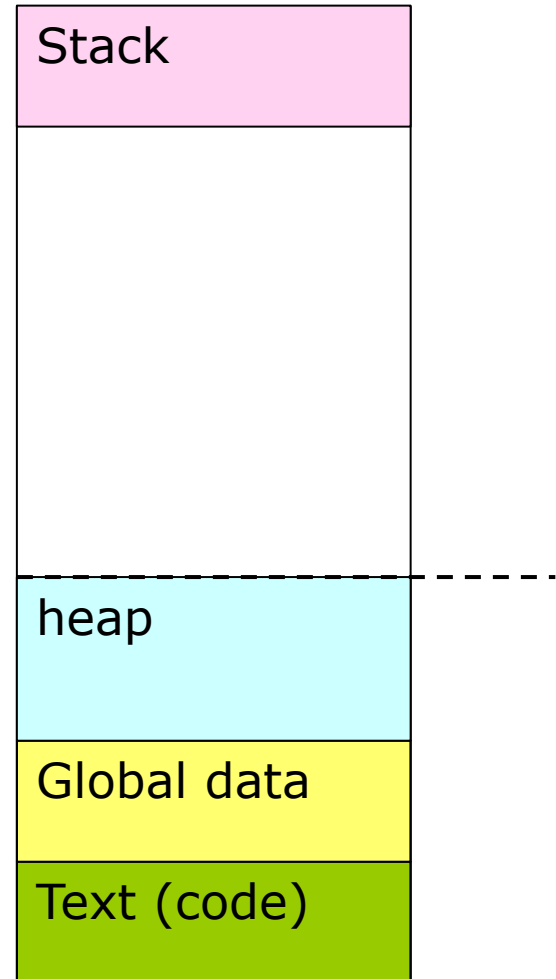
- เนื้อหานำมาจาก <https://www.geeksforgeeks.org/c/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc/>





malloc() implementation

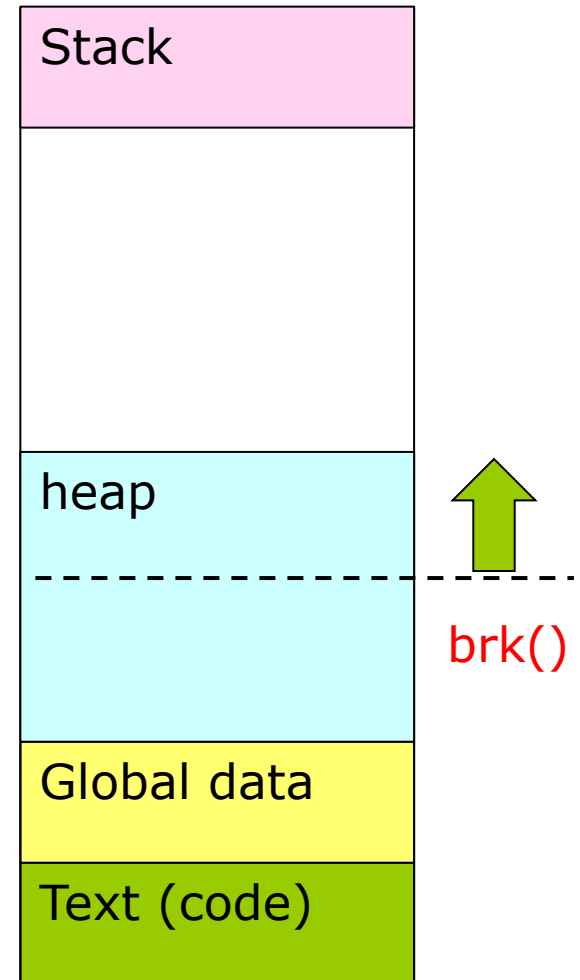
- malloc() เป็น standard library function มันเริ่มต้นขอพื้นที่ heap memory ก่อนใหญ่มาจาก kernel หลังจากนั้น มันจะจัดสรรพื้นที่ๆขอมาให้โปรแกรมใช้ตามปริมาณที่ต้องการ จนกระทั่งพื้นที่ว่างในพื้นที่ๆขอมาหมด หรือหาพื้นที่ memory block ที่ติดกัน ให้โปรแกรมไม่ได้
- malloc() ลดการเรียกใช้ system call เพื่อขอ memory จาก kernel เพราะ system call มี context switching overheads





malloc() implementation

- เมื่อต้องการ memory เพิ่ม โดยเรียกใช้ **brk()** หรือ **mmap()** system call เพื่อเพิ่มขนาดการใช้งาน memory ของ Process ใน heap segment
- เมื่อมีการจัดสรรและ free memory block ใน heap หลายครั้งเมื่อ process ประมวลผลไปนานๆ อาจส่งผลให้เหลือ contiguous memory block (พื้นที่ติดกัน) น้อยลง ทำให้ malloc ต้องขยายขนาดของ heap



End of Chapter 12

